UNIVERSITÉ DE VERSAILLES ST-QUENTIN-EN-YVELINES

université PARIS-SACLAY

Programmation GPU

(APM 2026-2027)

TP noté

Floyd-Warshall sur GPU

durée : 3 h

adrien.roussel@cea.fr

Les objectifs de ce TP sont :

— écrire et lancer un premier kernel CUDA, et gérer la mémoire device de bout en bout ;

— analyser la nature des accès mémoire (coalescence) et mesurer un débit ;

— en bonus, exploiter la mémoire partagée (Floyd-Warshall par blocs).

Rendu. Une archive « NOM.PRENOM.tar.gz » de votre répertoire de travail, créée en fin de séance depuis « CODE » :

```
bash rendu.sh
```

Le script demande votre nom et votre prénom, crée le répertoire « NOM.PRENOM » et y rassemble les fichiers de rendu : votre « fw_gpu.cu » (créé au début de la section 1), « REPONSES.md » rempli, les sources C, C++ et CUDA et le « Makefile » ; il ne copie aucun exécutable et vérifie l'archive, dont il affiche le contenu, à contrôler avant l'envoi. Envoyez l'archive en pièce jointe à « adrien.roussel@cea.fr ». Les fichiers fournis (« fw.c », « fw_utils.h », « cuda_utils.h », « Makefile ») ne doivent pas être modifiés. Les questions de code (Q.2 à Q.5, Q.7, Q.10) sont évaluées dans « fw_gpu.cu », les autres dans « REPONSES.md ».

Consignes. L'usage d'assistants IA n'est pas autorisé ; les rendus sont analysés. Si une question vous bloque plus de 15 minutes, passez à la suivante : les sections 1 et 2 priment sur le bonus.

section	questions	points	temps conseillé
1. Premier noyau CUDA	Q.1 à Q.5	10	70 min
2. Accès coalescés	Q.6 à Q.8	6	40 min
3. Bonus : mémoire partagée	Q.9, Q.10	4	50 min

Le problème et l'algorithme de Floyd-Warshall

Le problème. Un graphe orienté à n sommets, aux arêtes de poids positifs, est représenté par sa matrice d'adjacence : $d[i][j]$ est le poids de l'arête $i \to j$, 0 si $i = j$, et INF (un point dans les affichages) marque l'absence d'arête. Le problème traité dans ce TP est celui des plus courts chemins entre tous les couples : calculer, pour chaque couple (i, j), la longueur du plus court chemin de i à j, INF s'il n'en existe pas.

L'algorithme. Floyd-Warshall résout ce problème en n passages successifs, la matrice étant mise à jour sur place. Au passage k, chaque case est mise à jour par la formule :

$$d_{ij} \leftarrow \min(d_{ij},\ d_{ik} + d_{kj}),$$

c'est-à-dire : passer par le sommet intermédiaire k raccourcit-il le chemin de i à j ? Après les n passages, $d[i][j]$ contient la longueur du plus court chemin de i à j. Le coût total est de l'ordre de n^3 mises à jour : les confier au GPU est l'objet de ce TP.

L'exemple du sujet. Le graphe déterministe à 4 sommets ci-dessous, construit par « graph_example() » dans « fw.c », sert de référence tout au long du TP : à gauche le graphe de base, à droite le résultat attendu après Floyd-Warshall. Sans option, « ./fw » calcule et affiche ces deux matrices ; votre version GPU devra reproduire exactement celle de droite.

0	3	10	·
·	0	4	8
·	·	0	1
·	·	·	0

graphe de base

0	3	7	8
·	0	4	5
·	·	0	1
·	·	·	0

plus courts chemins

Le code de base et votre environnement

Le code fourni. Le répertoire « CODE » contient la référence CPU « fw.c », deux headers et le script de rendu :

- « fw.c » : Floyd-Warshall sur CPU. Il ne contient que « floyd_warshall » (l'algorithme, la matrice étant mise à jour sur place) et « main » ; « make » le compile et il **ne doit pas être modifié** : c'est votre référence ;
- « fw_utils.h » : les utilitaires « graph_example() » (l'exemple déterministe ci-dessous), « graph_random(n, seed) » (graphe aléatoire reproductible), « print_graph(d, n) » (affichage, un point marquant l'absence d'arête) et « now_ms() » (horloge en millisecondes) ;
- « rendu.sh » : le script qui prépare l'archive de rendu (paragraphe « Rendu » ci-dessus) ;
- « cuda_utils.h » : des aides pour votre version GPU. Ce header n'est utilisé ni par « fw.c » ni par le « Makefile » : il est à votre disposition, incluez-le dans « fw_gpu.cu ». Il ne fournit que la vérification d'erreur et le chronométrage : ni allocation, ni copie, ni lancement, qui restent à écrire et doivent apparaître dans votre code rendu :

```
CUDA_CHECK(cudaMalloc(&d, octets));  /* arrete en cas d'erreur */

gpu_timer_t t;
gpu_timer_start(&t);
/* ... vos lancements et copies ... */
float ms = gpu_timer_stop(&t);       /* duree en millisecondes */
```

Votre environnement. La frontale du cluster (`romeo1.univ-reims.fr`) est en x86, les nœuds de calcul en ARM : compilez et exécutez sur un nœud réservé du cluster, après `module load cuda`. Les commandes de base :

```
srun --account r260073 --time 00:30:00 --mem 20G \
     --constraint armgpu -c72 -n1 --gpus-per-node 1 --pty /bin/bash
cp fw.c fw_gpu.cu                    # votre fichier de travail
nvcc -O3 -arch=sm_90 -o fw_gpu fw_gpu.cu
./fw                                 # reference CPU (exemple du sujet)
./fw -n 512                          # reference CPU, graphe aleatoire
./fw_gpu ...                         # votre version GPU
```

La première commande réserve un nœud GPU et y ouvre un shell interactif ; sans `--pty /bin/bash`, `srun` exécute directement la commande passée en argument. Les GPU de la salle sont des GH200 : architecture `sm_90`.

I Premier noyau CUDA : un thread par case

La fonction « floyd_warshall » de « fw.c » applique l'algorithme présenté en introduction. Pour un k fixé, les n^2 mises à jour du passage k sont indépendantes les unes des autres : c'est exactement le travail que l'on peut confier à un kernel, un thread par case. Cette section construit cette première version ; les suivantes l'analysent puis l'améliorent.

Étape 0 (à faire avant Q.1, aucun point). Copiez « fw.c » en « fw_gpu.cu », compilez-le avec `nvcc` (commande ci-dessus) et exécutez-le : c'est du C, il doit se comporter exactement comme « ./fw ». Vous venez de compiler votre premier programme CUDA.

Q.1: La boucle sur k ne peut pas être déplacée telle quelle dans le kernel. (a) Qu'est-ce que le passage $k+1$ lit qui vient d'être produit par le passage k ? (b) À quelle échelle « ___syncthreads() » synchronise-t-il les threads, et que manque-t-il pour synchroniser toute la grille en CUDA standard ? (c) Conclusion : où reste la boucle sur k, et qu'est-ce qui, dans votre programme, joue le rôle de barrière entre deux pas successifs ? (1 point, dans « REPONSES.md »).

Q.2: Toujours dans « fw_gpu.cu », écrivez la fonction hôte « fw_gpu(int *d, int n) », appelée depuis le « main » à la place de « floyd_warshall », et conservez « floyd_warshall » dans le fichier (elle sert à la question 4). À ce stade, elle prépare le terrain autour du kernel : allocation mémoire, transferts, dimensionnement de la grille. Critère de validation : à la sortie, « d » est identique à l'entrée, ce que l'affichage du programme suffit à constater. (2 points).

Q.3: Écrivez dans « fw_gpu.cu » le kernel « fw_step(int *d, int n, int k) », qui effectue le passage k de l'algorithme, puis complétez « fw_gpu » pour que, comme « floyd_warshall », elle travaille sur place : à la sortie, « d » contient le résultat. (3 points).

Q.4: Validez : votre version GPU doit retrouver exactement le résultat de la référence CPU, sur l'exemple du sujet comme sur des graphes aléatoires de tailles variées (au moins $n = 8$, 64 et 512). Le programme doit afficher un verdict explicite. (2 points).

Q.5: Chronométrez les deux versions, transferts compris pour le GPU. Reportez les temps pour $n = 512$, 1024 et 2048 dans « REPONSES.md » et votre constat en une phrase. (2 points).

II Accès coalescés

Cette section analyse les accès mémoire de votre noyau de la section 1 et en mesure les conséquences sur les performances.

Q.6: Pour le kernel « fw_step » que vous avez écrit, classez chacun de ses accès mémoire : coalescé, diffusé ou strided (diffusé : la même adresse pour tout le warp ; strided : un pas constant entre threads voisins). La réponse dépend de votre répartition de i et j : précisez-la. (2 points, dans « REPONSES.md »).

Q.7: Si votre version de la section 1 n'est pas coalescée, corrigez-la. Dans tous les cas, implémentez aussi la variante avec l'autre répartition et mesurez les deux à $n = 4096$: reportez les temps et l'écart dans « REPONSES.md ». (2 points).

Q.8: Le modèle de trafic du noyau naïf. (a) Dans le meilleur des cas, combien d'octets transitent par mise à jour ? (b) Combien d'opérations arithmétiques par mise à jour ? Déduisez l'intensité arithmétique du noyau : est-il limité par le calcul ou par la mémoire ? Justifiez à l'aide de la balance de votre GPU (rapport du pic de calcul au pic de bande passante, en flop par octet). (c) Quel débit maximal (Go/s) votre version coalescée peut-elle atteindre, et quel pourcentage du pic de bande passante de votre GPU atteignez-vous à $n = 4096$? (2 points, dans « REPONSES.md »).

III Bonus : Floyd-Warshall par blocs (mémoire partagée)

Le principe. Le noyau naïf relit d_{ik} et d_{kj} à chaque mise à jour : le trafic total est de l'ordre de $16\,n^3$ octets. La version par blocs le réduit : découpez la matrice en tuiles `TILE` × `TILE` (posez « #define TILE 32 » dans « fw_gpu.cu ») ; un côté compte nb tuiles, la dernière pouvant être incomplète, et la tuile (I, J) couvre les lignes $32I$ à $32I + 31$ et les colonnes $32J$ à $32J + 31$. Chaque tuile utile est ensuite chargée une seule fois en mémoire partagée, pour les 32 passages d'un même pas de bloc : ce facteur de réutilisation ne dépend pas de n, qui fixe seulement le nombre de tuiles et de pas.

Le pas de bloc. La boucle sur k reste côté hôte (cf. Q.1), mais par blocs de 32 : le pas de bloc K, pour $K = 0, 1, \ldots, nb - 1$, couvre les 32 passages k de $32K$ à $32K + 31$. Pendant un pas de bloc, chaque case (i, j) subit, pour chacun de ces passages, la mise à jour du passage k :

$$d_{ij} \;\leftarrow\; \min(d_{ij},\; d_{ik} + d_{kj})$$

Où sont les opérandes ? Pour une case de la tuile (I, J), les trois valeurs de la formule vivent dans des tuiles précises : l'accumulateur d_{ij} est dans la tuile elle-même ; l'opérande d_{ik} est dans la tuile (I, K), même rangée de tuiles, colonne du bloc ; l'opérande d_{kj} est dans la tuile (K, J), ligne du bloc, même colonne de tuiles. La tuile (I, J) ne peut donc être traitée qu'une fois les tuiles (I, K) et (K, J) finalisées pour le pas K, c'est-à-dire une fois qu'elles ont subi les 32 passages du pas : la traiter avant fausserait le résultat.

Dans quel ordre traiter les tuiles ? Pendant un pas de bloc, trois cas se présentent :

3	3	**2**	3
3	3	**2**	3
2	**2**	**1**	**2**
3	3	**2**	3

— la tuile pivot (K, K), à l'intersection de la ligne et de la colonne K, ne lit qu'elle-même : phase **1**, un Floyd-Warshall 32×32 autonome (les intermédiaires k du pas sont précisément les sommets du pivot) ;
— les tuiles de la ligne K et de la colonne K lisent, en plus d'elles-mêmes, le pivot, qui leur fournit l'opérande manquant : phase **2** ;
— toutes les autres tuiles lisent, en plus d'elles-mêmes, les tuiles (I, K) et (K, J) de leur rangée et de leur colonne : phase **3**.

Le numéro de phase est ainsi le nombre de tuiles distinctes lues : une, deux, puis trois.

L'algorithme. Pour chaque pas de bloc K : la phase **1** applique les 32 passages du pas à la seule tuile pivot (K, K) ; la phase **2** les applique aux tuiles de la ligne K et de la colonne K ; la phase **3**, à toutes les autres. Chaque tuile se relit elle-même pendant ses 32 passages. Au sortir de la phase **3**, la matrice est exactement celle de la version naïve après le passage $32K + 31$.

Mise en œuvre. Un bloc de threads traite une tuile : il charge en mémoire partagée la tuile et ses voisines utiles, y déroule les 32 passages du pas, puis restitue la tuile en mémoire globale. Chaque phase est un lancement de kernel distinct ; la question suivante demande pourquoi.

Q.9: Pourquoi exactement trois lancements par pas de bloc K, plutôt qu'un seul qui ferait tout ? Quel risque concret y a-t-il à fusionner des phases en un seul lancement ? (1 point, dans « REPONSES.md »).

Q.10: Implémentez la version par blocs : les trois kernels « fw_phase1_kernel », « fw_phase2_kernel », « fw_phase3_kernel » (tous de signature « (int *d, int n, int K) ») et l'orchestration « fw_blocked(int *d, int n) », sur le modèle de votre « fw_gpu ». Votre validation CPU / GPU doit rester verte, y compris pour des tailles non multiples de 32 (par exemple $n = 33$) ; mesurez ensuite l'accélération par rapport à votre version naïve coalescée et reportez-la dans « REPONSES.md ». (3 points).

Annexe : rappels

— Chronométrage : « gpu_timer_start » et « gpu_timer_stop » (« cuda_utils.h ») pour le GPU, « now_ms() » (« fw_utils.h ») pour le CPU ; durées en millisecondes.
— « min(a, b) » est utilisable dans un kernel.
— Vérifiez chaque appel avec « CUDA_CHECK » (« cuda_utils.h »).
— Copier vers le cluster : `scp fichier romeo1.univ-reims.fr:` ; récupérer depuis le cluster : `scp romeo1.univ-reims.fr:fichier .` (au besoin, `login@romeo1.univ-reims.fr`).